

**LAPORAN PRAKTIKUM
APLIKASI BERBASIS PLATFORM
MODUL 1 – 3**



**Disusun oleh:
Didik Weka Pratama
2311102285
S1 IF-11-04**

**Dosen Pengampu:
Cahyo Prihantoro, S.Kom., M.Eng.**

**PROGRAM STUDI S1 INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025/2026**

BAB 1 – RUNNING MODUL (Instalasi dan Konfigurasi Lingkungan Pengembangan)

Tujuan Praktikum

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

- Melakukan instalasi dan konfigurasi Git sebagai sistem pengontrol versi.
- Melakukan instalasi JDK (Java Development Kit) dan mengatur environment variable.
- Melakukan instalasi Flutter SDK dan mengintegrasikannya ke dalam PATH sistem.
- Melakukan instalasi Android Studio beserta komponen SDK yang diperlukan.
- Melakukan instalasi Visual Studio Code beserta ekstensi Flutter dan Dart.

1.1. Git

Git adalah salah satu sistem pengontrol versi (Version Control System) pada proyek perangkat lunak yang diciptakan oleh Linus Torvalds. Pengontrol versi bertugas mencatat setiap perubahan pada file proyek yang dikerjakan oleh banyak orang maupun sendiri. Git dikenal juga dengan distributed revision control (VCS terdistribusi), artinya penyimpanan database Git tidak hanya berada dalam satu tempat saja.

Untuk melakukan instalasi Git pada komputer, lakukan langkah-langkah berikut:

1. Buka link <https://git-scm.com/download/win> untuk mengunduh Git.
2. Klik dua kali pada file yang sudah diunduh.
3. Muncul informasi lisensi Git, klik Next untuk melanjutkan.
4. Pilih komponen yang akan dipasang, lalu klik Next.
5. Pilih editor default yang akan digunakan oleh Git (Nano atau Vim).
6. Pilih opsi ketiga agar Command Prompt dapat mengenali Git dan beberapa perintah UNIX.
7. Gunakan opsi default hingga tahap install, lalu klik Install.
8. Pastikan Git sudah terinstall dengan menjalankan perintah: `git --version` pada Command Prompt.

Setelah instalasi selesai, lakukan konfigurasi awal dengan perintah berikut:

```
git config --global user.name "Nama Anda"  
git config --global user.email email.anda@contoh.com
```

1.2. Instalasi JDK

JDK (Java Development Kit) merupakan perangkat yang digunakan untuk melakukan proses kompilasi dari kode Java ke bytecode yang dapat dimengerti dan dijalankan oleh JRE (Java Runtime Environment). Berikut merupakan langkah-langkah instalasi JDK:

1. Buka link <https://www.oracle.com/java/technologies/javase-jdk15-downloads.html> untuk mengunduh JDK sesuai dengan sistem operasi yang digunakan.
2. Buka file instalasi yang telah diunduh, lalu klik Next.
3. Pilih folder path tempat menyimpan instalasi JDK (disarankan menggunakan path default), lalu klik Next.
4. Tunggu proses instalasi hingga selesai, lalu klik Close.
5. Akses folder instalasi JDK dan salin path-nya.
6. Buka menu Search dan cari 'Environment Variables', lalu buka aplikasi tersebut.
7. Pada tab Advanced, klik 'Environment Variables'.
8. Pada bagian User Variables, klik Path dan tambahkan variabel baru dengan nama JAVA_HOME dan isi dengan path JDK yang telah disalin.
9. Lakukan hal yang sama pada bagian System Variables.
10. Klik OK untuk menyelesaikan proses konfigurasi.

1.3. Instalasi Flutter SDK

Sebelum melakukan instalasi Flutter, perlu disiapkan beberapa tools yang dibutuhkan dalam pengembangan aplikasi. Flutter SDK mendukung berbagai sistem operasi dengan persyaratan minimum yang berbeda.

1.3.1. Persyaratan Minimum

a. Windows

- Sistem Operasi Windows 7 SP1 atau lebih baru (64-bit), x86-64 based.
- Ruang penyimpanan minimal 1.64 GB (tidak termasuk IDE dan tools lainnya).
- Windows PowerShell 5.0 atau versi terbaru.
- Git for Windows 2.x dengan opsi 'Use Git from the Windows Command Prompt'.

b. macOS

- Sistem Operasi Mac OS 64-bit.
- Ruang penyimpanan minimal 2.8 GB.
- Tools: bash, curl, git 2.x, mkdir, rm, unzip, which.

c. Linux

- Sistem Operasi Linux 64-bit.
- Ruang penyimpanan minimal 1.8 GB.

- Tools: bash, curl, git 2.x, mkdir, rm, unzip, which, xz-utils.

1.3.2. Langkah Instalasi Flutter SDK

1. Unduh Flutter SDK pada <https://flutter.dev/docs/development/tools/sdk/releases>. Pilih versi stabil terbaru yang sesuai dengan sistem operasi.
2. Ekstrak berkas ZIP dan tempatkan folder flutter pada lokasi instalasi yang diinginkan, misalnya C:\Development. Catatan: Jangan pasang Flutter di direktori seperti C:\Program Files.
3. Temukan berkas flutter_console.bat di dalam direktori flutter tersebut dan klik dua kali untuk menjalankan Flutter Console.
4. Update PATH: Buka Environment Variables, lalu tambahkan path folder flutter\bin ke dalam variabel PATH agar perintah Flutter dapat digunakan dari mana saja.
5. Verifikasi instalasi dengan menjalankan perintah flutter doctor.

Perintah flutter doctor digunakan untuk mengecek kelengkapan framework Flutter yang akan digunakan, termasuk versi, Android SDK, perangkat yang terhubung, dan sebagainya.

```
flutter doctor
```

1.4. Instalasi Android Studio

Android Studio merupakan IDE resmi untuk membangun aplikasi berbasis Android. Berikut merupakan persyaratan dan langkah instalasinya:

1.4.1. Persyaratan Minimum

- Microsoft Windows 7/8/10 (64-bit).
- RAM minimal 4 GB, disarankan 8 GB.
- Ruang disk minimal 2 GB, disarankan 4 GB (500 MB untuk IDE + 1.5 GB untuk Android SDK dan emulator).

1.4.2. Proses Instalasi

Unduh Android Studio terbaru pada <https://developer.android.com/studio> kemudian ikuti langkah instalasi berikut:

9. Jalankan file installer (.exe) dengan klik dua kali.
10. Pastikan Android Virtual Device ter-checklist agar dapat menggunakan emulator.
11. Pilih folder instalasi, lalu klik Next.
12. Pilih folder Start Menu, lalu klik Install.
13. Tunggu proses instalasi hingga selesai, lalu klik Next dan Finish.
14. Pada peluncuran pertama, pilih 'Do not import settings' jika belum pernah menginstall Android Studio sebelumnya.
15. Pilih Installation Type Standard (direkomendasikan), lalu klik Next.

16. Pilih tema tampilan yang diinginkan, lalu klik Next.
17. Klik Finish untuk mengunduh komponen yang dibutuhkan.
18. Setelah instalasi selesai, buat project baru dengan memilih 'New Project', lalu pilih 'Empty Activity'.
19. Masukkan nama aplikasi dan pilih bahasa pemrograman Java, lalu klik Finish.

1.5. Instalasi SDK Android

SDK (Standard Development Kit) merupakan kumpulan dari beberapa alat, komponen, dan platform untuk mengembangkan aplikasi berbasis Android. SDK wajib ada pada Android Studio. Berikut cara instalasinya:

- Klik menu Tools pada menu bar Android Studio, lalu pilih SDK Manager.
- Pada jendela SDK Manager, pilih versi Android SDK yang ingin digunakan.
- Centang komponen yang diperlukan, lalu klik Apply dan tunggu proses pengunduhan selesai.
- Klik OK untuk menutup jendela SDK Manager.

1.6. Instalasi Visual Studio Code

Visual Studio Code (VS Code) merupakan code editor ringan yang sangat populer di kalangan pengembang. Untuk menginstall VS Code, unduh installer dari <https://code.visualstudio.com/> kemudian jalankan file installer dan ikuti petunjuk yang muncul. Setelah selesai, VS Code siap digunakan.

1.7. Instalasi Extension Visual Studio Code

Untuk mendukung pengembangan aplikasi Flutter di VS Code, dua ekstensi wajib dipasang, yaitu ekstensi Flutter dan Dart. Langkah instalasinya adalah sebagai berikut:

- Buka VS Code dan klik ikon Extensions di sidebar (atau tekan Ctrl+Shift+X).
- Pada kotak pencarian, ketik 'Flutter' dan klik tombol Install pada ekstensi Flutter by Dart Code.
- Ekstensi Dart akan otomatis terpasang bersamaan dengan ekstensi Flutter.
- Verifikasi instalasi dengan membuka Command Palette (Ctrl+Shift+P) dan mengetik 'Flutter: New Project'.

BAB 2 – PENGENALAN FLUTTER

Tujuan Praktikum

- Mahasiswa mampu memahami konsep dasar Flutter dan arsitekturnya.
- Mahasiswa dapat mengimplementasikan aplikasi Hello World pertama menggunakan Flutter.

2.1. Apa Itu Flutter

Flutter ditulis menggunakan bahasa C, C++, dan Dart dengan Google's Skia Graphics Engine untuk user interface. Engine yang digunakan pada Flutter dikenal luas karena juga dipakai pada produk seperti Google Chrome, Chrome OS, Chromium OS, Mozilla Firefox, Mozilla Thunderbird, Android, Firefox OS, dan sekarang Flutter.

Flutter berjalan menggunakan Dart Virtual Machine (VM) di sistem operasi Windows, Linux, dan macOS. Dart VM menggunakan kompilasi kode just-in-time (JIT) yang menyediakan fitur hot-reload untuk menghemat waktu pengembangan.

Flutter API menggabungkan beberapa widget sesuai dengan kebutuhan pada aplikasi. Konsep widget tree pada dasarnya adalah implementasi widget yang di dalamnya terdapat widget lain yang mewakili komponen antarmuka. Widget bisa stateless atau stateful, dan perbedaan dari keduanya sesuai dengan status widget itu sendiri. Hal tersebut berguna untuk membantu mengelola status aplikasi.

2.2. Arsitektur Flutter

Selain konsep inti yang telah dibahas, Flutter memiliki arsitektur dasar yang dapat diterapkan pada aplikasi untuk mengelola statusnya dengan mudah. Arsitektur yang digunakan pada Flutter disebut Business Logic Component (BLOC).

Pada dasarnya, arsitektur BLOC bekerja melalui pendekatan ketika terjadinya suatu event dan handle perubahan state pada aplikasi. BLOC merupakan pendekatan yang baik untuk memisahkan logika bisnis dari antarmuka pengguna. Ide inti yang digunakan pada arsitektur BLOC adalah simplicity (kesederhanaan), scalability (skalabilitas), dan testability (kemampuan pengujian), dan semua tujuan ini dicapai dalam arsitektur BLOC.

2.3. Hello World Pada Flutter

Pada pengenalan Flutter ini, akan dibuat program Hello World sebagai permulaan. Berikut langkah-langkah pembuatannya:

20. Buka Visual Studio Code untuk membuat project Flutter.
21. Pastikan ekstensi Flutter dan Dart sudah terpasang.
22. Buat project Flutter melalui menu View → Command Palette → Flutter: New Project → Application → pilih folder penyimpanan → beri nama project → tekan Enter.
23. Setelah project selesai dibuat, buka file main.dart.
24. Hapus semua kode yang ada pada main.dart.
25. Tambahkan kode berikut untuk membuat Hello World:

```
// ignore_for_file: prefer_const_constructors
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: "Hello World",
      home: const MyHomePage(title: "Flutter Hello World Page"),
    );
  }
}

class MyHomePage extends StatefulWidget {
  const MyHomePage({Key? key, required this.title}) : super(key: key);
  final String title;

  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

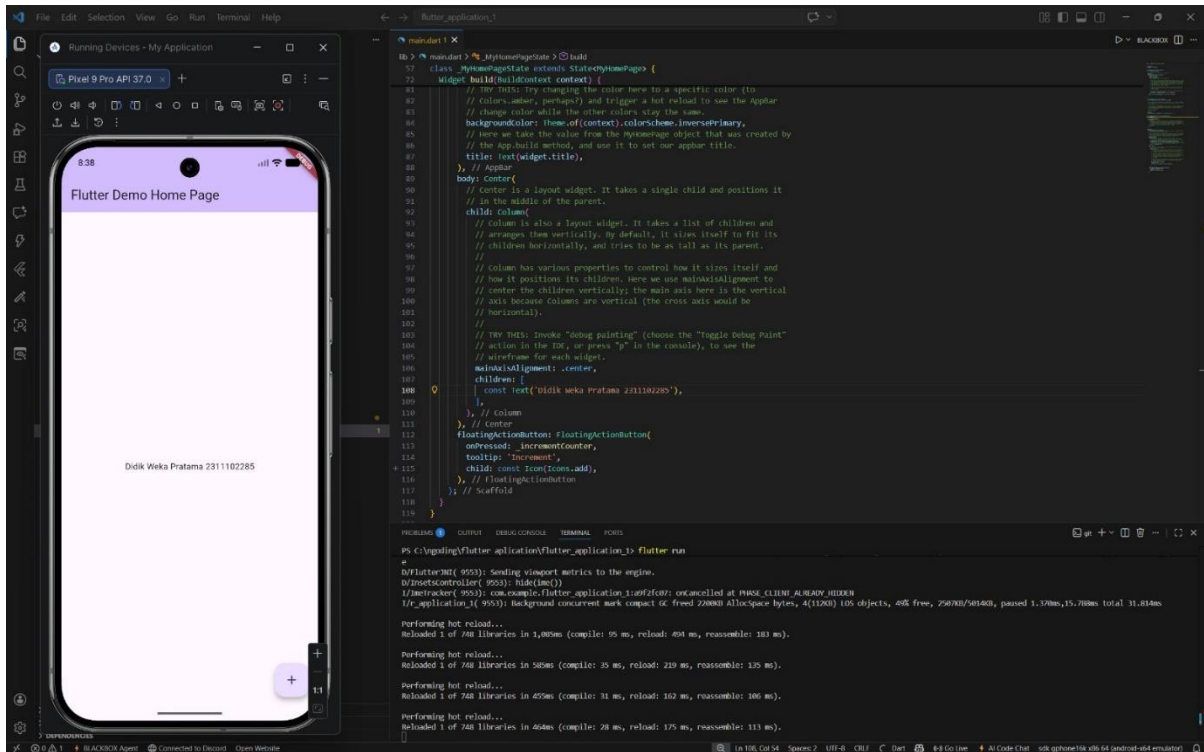
class _MyHomePageState extends State<MyHomePage> {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(widget.title),
      ),
```

```

    body: Center(
      child: Text(
        'Hello World',
      ),
    ),
  );
}
}

```

Kode di atas merupakan struktur dasar aplikasi Flutter. Kelas MyApp bertindak sebagai root aplikasi yang mengembalikan widget MaterialApp. Kelas MyHomePage merupakan widget stateful yang memiliki tampilan berupa Scaffold dengan AppBar dan konten berupa teks 'Hello World' yang ditampilkan di tengah layar.



BAB 3 – PENGENALAN DART

Tujuan Praktikum

- Mahasiswa mampu memahami konsep dasar bahasa pemrograman Dart.
- Mahasiswa dapat mengimplementasikan sintaks dasar Dart meliputi variabel, statement control, looping, list, dan fungsi dalam konteks pengembangan Flutter.

3.1. Pengenalan Dart

Untuk belajar Flutter, tidak diperlukan keahlian tinggi dalam bahasa Dart. Terdapat beberapa fundamental yang perlu dipelajari seperti variable, statement control, looping, array, fungsi, dan sebagainya. Karakteristik bahasa Dart mirip dengan bahasa C ataupun Java. Aturan penting dalam Dart adalah wajib menggunakan titik koma (;) di akhir setiap statement.

3.1.1. Variable

Untuk penggunaan variabel di Dart, terdapat beberapa cara, yaitu dengan var, type annotation, dan multiple variable. Berikut sintaksnya:

```
// var
var <variable_name>;
var <name> = <expression>;

// type annotation
<type> <variable_name>;
<type> <name> = <expression>;

// multiple variable
<type> <var1, var2...varN>;
```

Tipe data primitif yang tersedia di Dart antara lain:

- Integer – untuk bilangan bulat.
- Double – untuk bilangan desimal.
- String – untuk teks.
- Boolean – untuk nilai true/false.

3.1.2. Statement Control

Terdapat beberapa cara untuk mendeklarasikan statement control di Dart, yaitu if, if-else, if-else if, dan switch-case.

IF Statement

```
if(condition) {
    // statements
}
```

```
}
```

IF ELSE Statement

```
if(condition) {  
    // statements  
} else {  
    // statements  
}
```

IF ELSE IF Statement

```
if(condition1) {  
    // statement(s)  
}  
else if(condition2) {  
    // statement(s)  
}  
.  
.  
else if(conditionN) {  
    // statement(s)  
}  
else {  
    // statement(s)  
}
```

SWITCH CASE Statement

```
switch(expression) {  
    case value1: {  
        // statements  
    }  
    break;  
    case value2: {  
        // statements  
    }  
    break;  
    default: {  
        // statements  
    }  
    break;  
}
```

3.1.3. Looping

Secara umum, terdapat dua cara untuk melakukan looping di Dart, yaitu menggunakan for loop dan while loop.

For Loops

Gunakan for loop saat sudah diketahui berapa banyak perulangan akan dilakukan, misalnya melakukan perulangan sebanyak 10 kali dengan iterasi 1 tingkat.

```
for (initial_count_value; termination-condition; step) {  
    // statements  
}
```

While Loops

Gunakan while loop saat tidak diketahui kapan perulangan akan berhenti, misalnya meminta input angka hingga pengguna menginput tanda tertentu.

```
while (expression) {  
    // Statement(s) to be executed if expression is true  
}
```

3.1.4. List

Secara umum, kumpulan banyak data dalam satu variabel disebut array. Namun dalam bahasa Dart, hal ini disebut dengan list. List memiliki 2 tipe, yaitu Fixed Length List dan Growable List.

Fixed Length List

Tipe list ini memiliki panjang index yang tetap dan tidak dapat bertambah. Sintaksnya adalah sebagai berikut:

```
// Mendeklarasikan list  
var list_name = new List(initial_size);  
  
// Menginisialisasikan list  
list_name[index] = value;  
  
// Contohnya  
var newList = new List(3);  
newList[0] = 12;  
newList[1] = 13;  
newList[2] = 11;
```

Growable List

Gunakan growable list apabila memiliki banyak object yang tidak menentu jumlahnya atau terus bertambah. Sintaksnya adalah sebagai berikut:

```
// Mendeklarasikan list  
var list_name = new List();  
  
// Menginisialisasikan list
```

```
list_name[index] = value;

// Contohnya
var newList = new List();
newList.add(12);
newList.add(13);
newList.add(11);
```

3.1.5. Fungsi

Pada bahasa pemrograman yang mendukung Object Oriented Programming (OOP), fungsi atau prosedur memiliki peranan yang sangat penting. Untuk menghasilkan kualitas kode yang baik, programmer dapat menggunakan beberapa prinsip pemrograman yang umum seperti SOLID, KISS, YAGNI, dan sebagainya.

Semua prinsip tersebut menjunjung tinggi separation of concern, yang artinya setiap bagian kode memiliki tanggung jawabnya sendiri dan mengurangi sebanyak mungkin boilerplate code. Berikut sintaks dasar fungsi dalam Dart:

```
// Deklarasi fungsi dengan return type
<return_type> <function_name>(<parameters>) {
    // body function
    return value; // jika bukan void
}

// Contoh fungsi dengan return value
int tambah(int a, int b) {
    return a + b;
}

// Contoh fungsi tanpa return value (void)
void cetak(String pesan) {
    print(pesan);
}

// Memanggil fungsi
void main() {
    int hasil = tambah(5, 3);
    cetak("Hasil: $hasil");
}
```

Dengan menggunakan fungsi, kode menjadi lebih terstruktur, mudah dipahami, dan dapat digunakan kembali (reusable) di berbagai bagian program.